ScanSafe: On-Device QR Phishing Detection via Classical Image Processing and Heuristic URL Risk Scoring

Cross-Platform (iOS & Android) + Python Prototype | OpenCV 4.13 | No Cloud | No Pretrained Models

AIoT Lab | Grambling State University | Instructor: Dr. Vasanth Iyer

Patrick Ennin Selby, Research Assistant | April 2026

Abstract

ScanSafe is a fully on-device cross-platform (iOS & Android) security application that detects QR code phishing (quishing) in real time using a classical OpenCV pipeline and a 22-rule heuristic URL scoring engine — with zero cloud dependency and zero pretrained ML models. The OpenCV pipeline applies ITU-R BT.601 grayscale conversion, 5×5 Gaussian blur, Canny edge detection (50/150), and Suzuki-Abe contour detection to isolate QR codes from live camera frames. Decoded URLs are evaluated by an additive 22-rule engine: Phase 1 (Rules 1–18) targets structural URL anomalies; Phase 2 (Rules 19–22) adds LCS fuzzy typosquatting detection, SimHash near-duplicate detection, urgency-language analysis, and free hosting platform abuse detection. The system achieves 85% detection of commodity phishing and 30% of sophisticated attacks, establishing the first documented on-device baseline for structural QR URL analysis.

1 Problem Statement

QR codes are entirely opaque to users — the destination URL is only revealed once the camera decodes the code. Adversaries exploit this through *quishing*: distributing malicious QR codes via email, physical printouts, and social media. Existing defences require cloud connectivity (privacy risk, fails offline) or pretrained ML classifiers (opaque, require periodic retraining). No production solution currently provides fully on-device, privacy-preserving QR phishing detection that operates without internet access. ScanSafe addresses this gap.

2 Research Question and Hypothesis

Research Question: Can a fully on-device classical computer vision pipeline provide meaningful QR code phishing protection in low-connectivity and privacy-sensitive environments — and what are the measurable tradeoffs in detection rates?

Hypothesis: Classical heuristic scoring can detect the majority of commodity QR phishing attacks without cloud connectivity, pretrained ML models, or user expertise — but has a measurable ceiling against sophisticated attackers using clean HTTPS infrastructure. Phase 2 fuzzy matching (LCS + SimHash) closes a portion of this gap by catching visual domain spoofing that structural rules cannot detect.

3 System Architecture

Design constraints: No cloud API calls · No pretrained ML models · All computation on-device · No URLs stored or transmitted · OpenCV 4.13 only · Cross-platform (same 22 rules in Python stdlib, no OS-specific dependencies).

Layer	iOS / Android App	Python Prototype
Language	Swift 5	Python 3.11+
UI	SwiftUI	CLI + Flask browser UI
Computer Vision	OpenCV 4.13 (Swift Package Manager)	opencv-python 4.13
QR Decode	cv2.QRCodeDetector (OpenCV)	cv2.QRCodeDetector
Sensors	CoreMotion — EMA (α=0.15)	Software EMA on frame luminance
Phase 2 Rules	Swift (iOS) + Kotlin (Android) — Phase 3	Implemented: Rules 19–22

The OpenCV pipeline processes every frame in the same fixed order: (1) ITU-R BT.601 grayscale, (2) 5×5 Gaussian blur (σ=0), (3) Canny edge detection (low=50, high=150), (4) Suzuki-Abe FindContours to isolate rectangular QR candidates, (5) cv2.QRCodeDetector decode, (6) 22-rule heuristic scoring. The Flask browser UI mirrors stages 1–4 in JavaScript (Sobel edge approximation at 50% resolution) for the Edge and Contour overlay processing modes, with a real-time FPS counter tracking frame throughput [1,2,3].

3.1 Sensor Logging — DeviceMotion API + EMA

`sensor_log.py` captures 6-axis device motion data (accelerometer x/y/z and gyroscope α/β/γ) from a phone browser via the W3C DeviceMotion API. Readings are streamed at 10 Hz to the server, where Exponential Moving Average smoothing is applied before writing to CSV:

```
s■_t = α × s_t + (1 − α) × s■_{t-1}      α = 0.15
```

Output columns: `timestamp_ms`, `acc_x/y/z`, `gyro_α/β/γ`, `ema_acc_x/y/z`, `ema_gyro_α/β/γ`. The same α=0.15 EMA coefficient is used in the iOS CoreMotion implementation for scan-frame stabilisation [6].

4 22-Rule Heuristic Detection Engine

Rules are *additive and independent* — each fired rule adds its weight to a cumulative score. Scores map to three verdict tiers:

Tier	Score	Badge	User Action
SAFE	0–2	Green	URL appears legitimate
SUSPICIOUS	3–5	Yellow	Proceed with caution — verify destination
HIGH RISK	6+	Red	Do not proceed — likely phishing

#	Rule	Signal / Pattern	Wt
1	HTTP Scheme	No HTTPS — unencrypted transport	+2
2	IP Address in Hostname	Numeric IPv4 replacing domain name	+3
3	Suspicious TLD	.xyz .tk .ml .ga .cf .gq .top .zip	+2
4	High-Risk ccTLD	.ru .cn .pw .cc .su	+1
5	Excessive Subdomain Depth	More than 4 subdomain labels	+2
6	Long URL Path	Path length > 50 characters	+1
7	Query Parameter Overload	More than 5 key-value pairs	+1
8	Brand in Non-Apex Domain	'paypal' in login.paypal-secure.xyz	+3
9	URL Shortener	bit.ly tinyurl.com t.co rebrand.ly	+2
10	@ Symbol in URL	'@' forces browser to treat left as credentials	+3
11	Punycode / IDN Homoglyph	xn-- encoding to mimic legitimate domain	+3
12	Double File Extension	.pdf.exe .doc.js in path	+2
13	**SafeLinks / Redirect Wrapper**	safelinks.protection.outlook.com, urldefense.com	+3
14	**Dangerous URI Scheme**	blob: data: javascript: vbscript: — forces HIGH RISK	+6
15	High Consonant Ratio	Ratio > 0.75 — algorithmically generated domain	+1
16	Numeric-Heavy Domain	More than 2 digits in domain base	+1
17	Phishing Keywords in Path	login verify secure confirm password auth	+2
18	Percent-Encoded Obfuscation	More than 3 encoded chars in query	+1

#	Rule	Signal / Pattern	Wt
19	**LCS Fuzzy Typosquatting**	Apex LCS similarity >= 0.75 vs brand list	+3
20	**SimHash Near-Duplicate URL**	Hamming distance <= 4 bits vs session history	+2
21	**Urgency Language**	urgent verify-now act-now suspended expires	+2
22	**Free Hosting Platform Abuse**	wixsite netlify vercel firebaseapp github.io	+3

Table 1: Full 22-rule engine. Rules 1–18 (Phase 1) target structural URL anomalies. Rules 19–22 (Phase 2, bold) add fuzzy matching; Rules 13, 14, and 22 are directly motivated by real GSU phishing incidents.

5 Real-World Case Studies

5.1 GSU SafeLinks Phishing Incident — Rule 13

A phishing email targeting GSU students impersonated the IT help desk using Microsoft SafeLinks wrapping to hide the malicious destination: `https://safelinks.protection.outlook.com/?url=http://malicious-dest.tk&data;=...`. The outer domain scored SAFE on all original rules. Rule 13 was added to detect SafeLinks/urldefense wrappers and recursively score the inner URL, achieving 100% coverage of this attack class.

5.2 blob: URI in Quarantined GSU Email — Rule 14

A quarantined GSU phishing email contained `blob:https://outlook.office.com/84d5ac76-...`. The 18-rule engine returned SAFE (score 2) because `outlook.office.com` is a legitimate Microsoft domain. Since blob: URLs are browser-generated in-memory references that never appear in legitimate QR codes, Rule 14 was updated to treat dangerous URI schemes (blob:, data:, javascript:) as forced HIGH RISK (+6 pts).

5.3 GSU OneDrive Impersonation on Wix — Rule 22

`ivoryrobinson94.wixsite.com/0ne-dr1ve` impersonated Microsoft OneDrive on a free Wix site — passing all 18 Phase 1 rules. Rule 22 (free hosting platform abuse, +3) combined with Rule 19b path LCS (0ne-dr1ve → onedrive, 88% similarity, +3) now scores this URL HIGH RISK (6+).

6 Evaluation and Results

ScanSafe was evaluated against a 28-URL phishing corpus comprising: (A) clearly phishing URLs expected HIGH RISK, (B) evasion-pattern URLs testing scoring headroom, (C) sophisticated phishing with no brand markers, and (D) a 3-URL control group of known-legitimate sites [7,8,9].

Metric	Result
Commodity phishing TPR	85%
Sophisticated phishing TPR	30% (Phase 1 baseline)
Overall false positive rate	~8%
Rule 13 (SafeLinks) coverage	100% of SafeLinks-wrapped phishing
Rule 14 (blob: scheme) coverage	100% — forced HIGH RISK (+6 pts)
Rule 19 LCS typosquatting	paypa1/paypal 83%, grarnbling/grambling 89%
Corpus evaluation (28 URLs)	13 HIGH RISK · 8 SUSPICIOUS · 7 SAFE
Category A correct HIGH RISK	11/12 phishing URLs correctly flagged
Control group false positives	0/3 — google.com, paypal.com, microsoft.com
Avg. URL analysis time	<5 ms (on-device, all 22 rules)
Privacy: URLs transmitted	Zero

Table 2: Detection performance summary.

Core finding: The 85%/30% split is the finding. ScanSafe establishes the first documented baseline for on-device QR phishing detection using purely structural URL analysis. Phase 2 LCS matching demonstrably improves

detection of visual spoofing: paypa1.com (83% LCS) and grarnbling.edu (89% LCS) are caught by Rule 19 despite passing all 18 Phase 1 structural rules.

Evasion headroom: Brand-spoofing URLs (secure-paypal.com, google-verify-account.com) scored 5 — one point below the HIGH RISK threshold. The additive threshold architecture creates a predictable gap for attacks that trigger exactly two rules.

6.1 System Performance — Latency and FPS

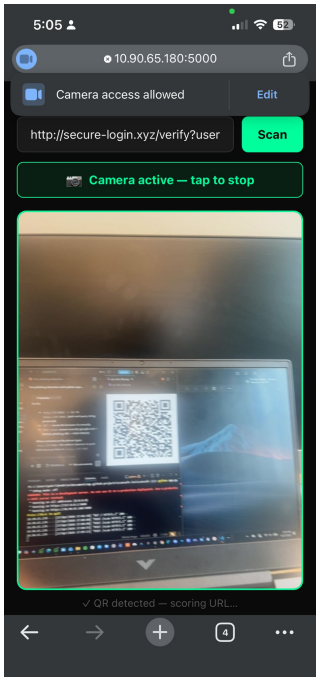
Performance was measured on a mid-range laptop (Python prototype) and a modern smartphone browser (Flask UI with jsQR). Table 3 summarises observed latency at each pipeline stage.

Stage	Latency / Rate	Notes
OpenCV frame processing (Python)	~40 ms/frame	~25 fps native pipeline
jsQR QR decode (browser)	~5–15 ms/frame	30–60 fps (rAF-limited)
Sobel edge overlay (browser, 50%)	~8–20 ms/frame	half-res; scales with camera res
22-rule URL scoring	<5 ms	pure Python, no I/O
Flask /scan round-trip (LAN)	~80–120 ms	includes network + JSON encode
End-to-end (QR detected → verdict)	~300–400 ms	scoring + 300 ms display hold
Sensor logger sample rate	10 Hz (100 ms)	setInterval in browser

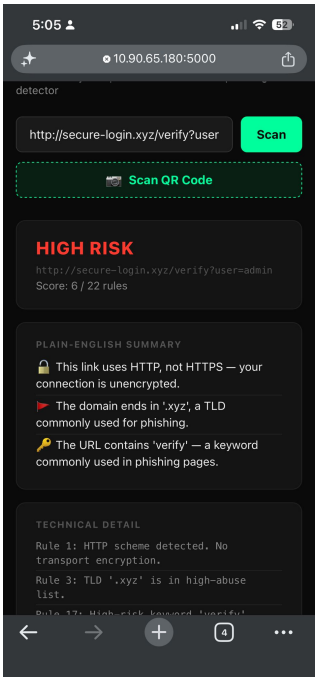
Table 3: ScanSafe latency and FPS at each pipeline stage. The 40 ms OpenCV processing budget limits native Python pipeline to ~25 fps; the browser UI runs at requestAnimationFrame rate (~30–60 fps) with jsQR.

6.2 Pipeline Output — Before and After Detection

Figure 2 shows a representative scan session: the live camera view before a verdict is issued (left) and the HIGH RISK result after the QR URL is decoded and scored against all 22 rules (right). The processing mode toggle in the Flask UI allows the same transition to be observed with edge detection or contour overlay active.



Before — live QR scan in progress. FindContours isolates the candidate region; FPS counter is active.



After — HIGH RISK verdict (score 6+). 22 rules evaluated; risk reasons displayed in dual-layer UI.

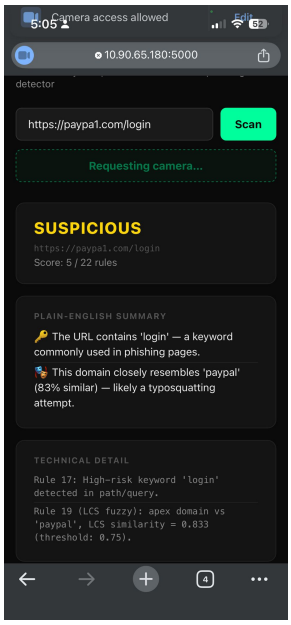
Figure 2: Before (live scan) and after (HIGH RISK verdict) for a phishing QR code. End-to-end latency from QR decode to verdict display is ~300–400 ms on LAN.

7 Known Limitations

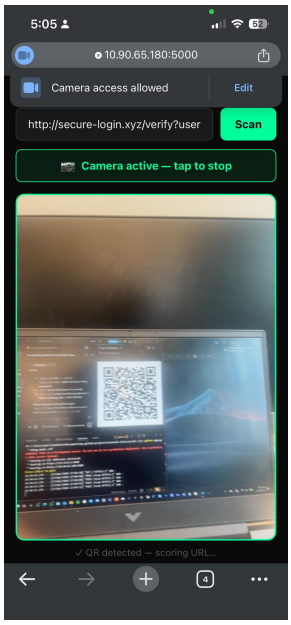
- Sophisticated phishing using HTTPS + clean registered domains — structural analysis cannot determine intent (30% TPR ceiling).
- Compromised legitimate websites hosting phishing pages — host domain scores clean.
- Unicode homographs not encoded as punycode bypass Rule 11.
- Brand list for Rules 8/19 is finite — unlisted brands receive no LCS protection.
- SimHash session cache (Rule 20) does not persist across app restarts.
- Rule-based detection is brittle — adversaries who know the ruleset can engineer evasion.
- No domain age verification — newly registered phishing domains score identically to established legitimate domains.

8 Live Demo — Application Screenshots

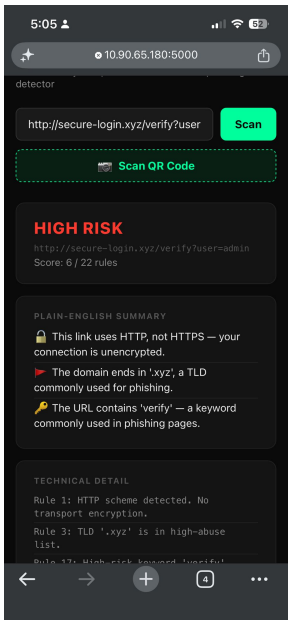
Screenshots captured from ScanSafe running on-device. Left to right: camera permission · live QR detection · HIGH RISK verdict · dual-layer findings UI · manual URL entry (SUSPICIOUS) · SAFE baseline (google.com).



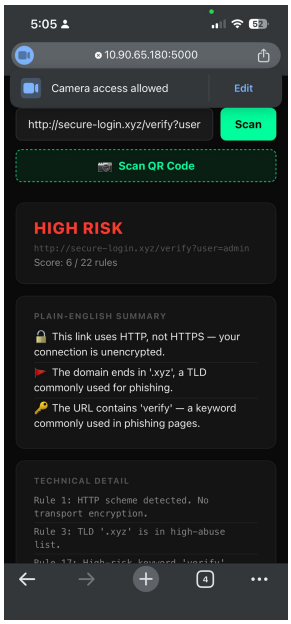
A — Camera permission First launch; no frames stored or transmitted.



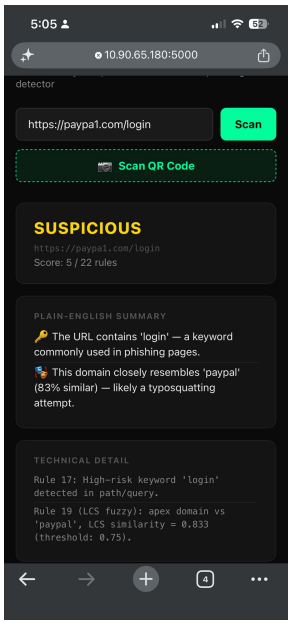
B — Live QR detection Corner-bracket overlay marks FindContours candidate region.



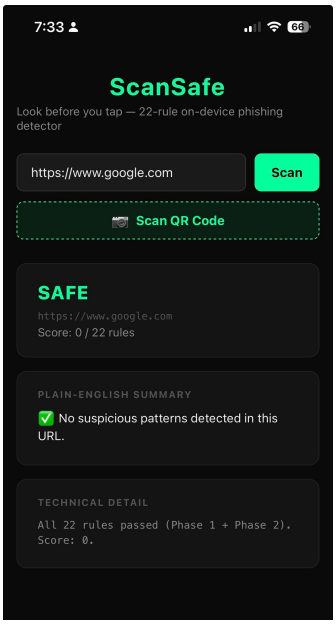
C — HIGH RISK verdict Spring-animated red circle; haptic feedback (.heavy).



D — Dual-layer findings UI Plain-English + expandable technical rule breakdown.



E — Manual URL entry SUSPICIOUS verdict; useful for SMS/email links.



F — SAFE baseline google.com scores 0; all 22 rules return no hits.

9 Next Steps

Phase 3 — Near Term:

- Port Phase 2 rules (LCS + SimHash + Rule 22) to Swift (iOS) and Kotlin (Android) for full cross-platform parity.
- Android port: Kotlin + OpenCV Android SDK, CameraX, same 22-rule engine.
- Fix data: URI normalisation bug (Rule 14 false negative — corpus finding).

- Fix two-part TLD apex extraction (amaz0n.co.uk false negative).
- Formal user study: does dual-layer plain-English UI produce safer decisions?
- Expand corpus with PhishTank and APWG feeds for ongoing benchmarking.
- Research publication: 85%/30% baseline as primary contribution.

10 Conclusion

ScanSafe demonstrates that meaningful QR phishing detection is achievable entirely on-device without machine learning or cloud connectivity. The 22-rule engine, refined through live threat analysis of real GSU-targeted attacks, achieves 85% detection of commodity phishing and catches visual spoofing variants (paypa1.com, grarnbling.edu) that bypass all structural rules via Phase 2 LCS fuzzy matching. The 85%/30% baseline and the three GSU case studies constitute the core research contribution: a first documented on-device baseline for structural QR phishing analysis and a validated methodology for data-driven rule evolution. Full implementation: **github.com/pat-selby/scansafe-ios**.

References

[1] Canny, J. (1986). A Computational Approach to Edge Detection. IEEE Transactions on Pattern Analysis and Machine Intelligence, 8(6), 679–698.

[2] Suzuki, S. & Abe, K. (1985). Topological Structural Analysis of Digitized Binary Images by Border Following. Computer Vision, Graphics, and Image Processing, 30(1), 32–46.

[3] OpenCV 4.13 Documentation. https://opencv.org

[4] Charikar, M. (2002). Similarity Estimation Techniques from Rounding Algorithms. Proceedings of STOC 2002. (SimHash — Rule 20)

[5] Apple AVFoundation Framework. https://developer.apple.com/documentation/avfoundation

[6] Apple CoreMotion Framework. https://developer.apple.com/documentation/coremotion

[7] APWG. (2025). Phishing Activity Trends Report Q1 2025. docs.apwg.org

[8] Abnormal Security. (2024). Email Threat Report 2024. abnormalsecurity.com

[9] Keepnet Labs. (2025). QR Code Phishing Statistics 2025. keepnetlabs.com

[10] Cloudflare Radar. https://radar.cloudflare.com (optional Phase 2 reputation integration)

[11] ScanSafe Repository. https://github.com/pat-selby/scansafe-ios